

SECURITY AUDIT

Symbiotic

Async/Account

FINAL REPORT

July 2026

BAILSEC.IO



Disclaimer:

Security assessment projects are time-boxed and reliant on information provided by the client, its affiliates, or its partners. The findings in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase, and apply only to the specific commit hash, version, or deployment reviewed at the time of the audit. Any subsequent modification, redeployment, or upgrade is outside the scope of this report and may invalidate its findings.

The audit does not constitute investment, legal, financial, regulatory, or tax advice, nor an endorsement of any project or team. It is based on a limited review of materials provided and is delivered on an "as-is," "where-is," and "as-available" basis. Use of blockchain technology is subject to unknown risks and flaws. The scope of this assessment is technical and contract-level; unless explicitly stated otherwise, it does not cover economic or game-theoretic risks, MEV, oracle manipulation, governance attacks, centralization or admin-key risks, off-chain components, front-end code, or the behavior of third-party protocols, dependencies, libraries, or services interacted with by the audited code.

We make no warranties, express or implied, regarding the accuracy, reliability, completeness, or availability of the report or any associated services, and disclaim all implied warranties including merchantability, fitness for a particular purpose, and non-infringement. We assume no responsibility for any product, service, software, code, or material referenced by, linked to, or accessible through the report.

This report is prepared solely for the contract owner and may not be relied upon by any third party. The contract owner is responsible for making their own decisions based on the report and should seek additional professional advice as needed. To the maximum extent permitted by law, our total aggregate liability arising from or related to this audit shall not exceed the fees actually paid for the engagement. The contract owner agrees to indemnify and hold harmless the audit firm and its personnel from any claims, damages, expenses, or liabilities arising from use of or reliance on the report or the audited smart contract.

This disclaimer and any dispute arising from the audit shall be governed by the laws of Wyoming (USA), without regard to its conflict-of-laws principles. By engaging in this audit, the contract owner acknowledges and agrees to these terms.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Symbiotic – Async/Account – Audit Report
Website	symbiotic.fi
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/symbioticfi/core-mirror
Resolution 1	

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High						
Medium	4	1		2		1
Low	12	1	1	6		4
Informational	11	2		8		1
Governance						
Total	27	4	1	16		6

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

3.1 Scope 1

Project	Symbiotic – Async/Account – Audit Report
Github repository	https://github.com/symbioticfi/core-mirror/blob/8fbde1bccc4400fc952f1484478209144dad8f06/src/contracts/adapters/Il-adapter/MidasAccount.sol
Resolution 1	https://github.com/symbioticfi/core-mirror/blob/32ade1b36ccc2472668382723c39e4eb97a3cced/src/contracts/adapters/Il-adapter/MidasAccount.sol

CutoffMidasAccount

The **CutoffMidasAccount** contract is a Midas redemption account variant that groups pending redemption requests into monthly cutoff cohorts instead of pricing them continuously. Each request is stamped with the bucket active at submission time, and once a cohort's cutoff date passes, all requests in that bucket freeze at the last Chainlink price printed before the cutoff rather than continuing to track the live oracle rate. It combines **MidasAccount**, which handles the underlying redemption-vault plumbing, with **CutoffAccount**, which supplies the generic bucket/timestamp mixin.

Invariants

- Every entry in **requestToBucket** corresponds to the **currentBucket()** value at the time that request was submitted.
- New redemption requests are only submitted outside the owner bypass when within **PRE_CUTOFF_WINDOW** of the next cutoff and past **COOLDOWN** since the last request.

Issue_01	<code>bucketToTimestamp</code> month-boundary corruption when <code>CUTOFF_DAY > 28</code>
Severity	Medium
Description	In <code>bucketToTimestamp</code>, the starting timestamp of a bucket is computed. However, if the <code>CUTOFF_DAY</code> is greater than 28, then erroneous timestamps are calculated for all months that do not have <code>CUTOFF_DAY</code> number of days. (February for 29, multiple months for 31). Say the initial timestamp is set to 31st January 2026, 00:00. Then, for bucket = 2, <code>bucketToTimestamp</code> calculates as follows: $month = 1 + 2 - 2 = 1$ $date_year = 2026$ $date_month = 2$ $date_day = 31$ $date_hour = 0$ But calling <code>DateTimeLib.dateTimeToTimestamp(2026,2,31,0,0)</code> returns 3rd March instead. Thus, there is a discrepancy and bucket boundaries overlap during the period 1-3rd March.
Recommendations	Ensure <code>CUTOFF_DAY < 28</code> in the constructor.
Comments / Resolution	Fixed by ensuring <code>CUTOFF_DAY <= 28</code> in the constructor.

Issue_02	mGLOBAL sync can revert on sub-minimum balances or Midas access-control failures, blocking finalization and swaps
Severity	Medium
Description	CutoffMidasAccount requests redemption whenever the sync path decides a request should be submitted and the account has a positive mGLOBAL balance. It does not check whether the balance satisfies Midas minimum redemption requirements before calling the Midas redemption vault. The Midas redemption request can revert when the balance is below the minimum amount, when the redemption function is paused, or when access-control checks fail. Because sync performs request finalization and request submission in the same transaction, a submission revert also rolls back any finalization that happened earlier in the same call. This also affects swaps. LiquidLaneAdapter transfers mGLOBAL into the account and then calls sync inline. If sync reverts because the resulting balance cannot be submitted to Midas, the whole swap reverts.
Recommendations	Check the Midas minimum redemption amount before attempting to submit a request. If the balance is below the minimum, skip request creation but still allow finalization of existing requests.
Comments / Resolution	Acknowledged.

Issue_03	Permissionless sync can grief real mGLOBAL redemptions
Severity	Low
Description	<code>CutoffMidasAccount.sync()</code> is permissionless. During the final cooldown period before a monthly cutoff, anyone can send a small redeemable amount of mGLOBAL to the account and call <code>sync()</code>. This creates a small redemption request and updates <code>lastRequestTimestamp</code>. If a real swap later sends a much larger mGLOBAL amount to the account before the cutoff, the adapter's <code>sync()</code> call cannot create a new redemption request because the cooldown is still active. The swap still pays the user, but the larger mGLOBAL balance remains idle in the account. As a result, that larger amount can miss the current monthly redemption cohort and be delayed until the next cycle unless the owner manually calls <code>sync()</code> before the cutoff.
Recommendations	Consider making the sync function admin-only, or documenting and acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_04	Round pricing discrepancy
Severity	Low
Description	The documentation states that “<i>pending requests compound until the cohort pricing date, then freeze at the first vault-feed print at/after it.</i>” This is inaccurate. In the current implementation, the cohort price is set at the last valid price before the end of the cohort, not the first price available after the end. This is evident from the loop shown below. <pre>uint48 nextBucketTimestamp = bucketToTimestamp(requestToBucket[requestId] + 1); while (timestamp >= nextBucketTimestamp) { - -roundId... // last oracle price of the cohort</pre>
Recommendations	Consider fixing the comment to reflect the price recording behaviour.
Comments / Resolution	Fixed by changing the comment.

Issue_05	Unbounded backward round walk can be gas-intensive
Severity	Low
Description	When valuing pending assets, the contract iterates over multiple chainlink rounds to find the accurate price, working backwards. <pre> while (timestamp >= nextBucketTimestamp) { [, answer,, timestamp,] = AggregatorV3Interface[aggregator].getRoundData[--roundId]; } </pre> Even within a single round, this loop can walk thousands of iterations for a feed that updates frequently, and if the cohorts are long. Thus, calling this function can become very gas-intensive, since there is no upper bound for the cost.
Recommendations	Consider having the admins cache the roundIds for pricing to fix the gas-cost of the function.
Comments / Resolution	Acknowledged.

Issue_06	Historical round used without staleness/validity checks
Severity	Low
Description	In <code>_pendingAssets</code>, the contract uses the historical price from the Chainlink aggregator directly to price the tokens undergoing redemption. The issue is that the raw price is used without any checks. The price could have been unupdated for a long time, and thus could have been too stale to use as the redemption price for the cohort. Chainlink documents that a zero <code>updatedAt</code> means the round is incomplete and must not be used. The project's own <code>ChainlinkPriceFeed</code> library already treats <code>updatedAt == 0</code> as invalid data. This path calls the aggregator directly and skips that check. The <code>Midas</code> vault oracle implements a bound check in its <code>getDataInBase18</code> function, which is also ignored here.
Recommendations	Consider adding a bound check for the historical oracle price, and ensure that the price being used has updates before and after it within a staleness threshold. Also, reject rounds with <code>updatedAt=0</code>, indicating incomplete rounds.
Comments / Resolution	Partially fixed by adding an <code>updatedAt</code> check.

Issue_07	CutoffMidasAccount skips Chainlink rounds published exactly at bucket cutoff
Severity	Low
Description	<code>CutoffMidasAccount._pendingAssets</code> prices pending requests by walking backward from <code>latestRoundData</code> until it finds a round with <code>updatedAt</code> strictly before <code>bucketToTimestamp(requestToBucket + 1)</code>. Monthly cutoff buckets are fixed on a calendar schedule, but Chainlink rounds are event-driven and may land at any second. Because the loop uses a strict less-than boundary, a round whose <code>updatedAt</code> equals the cutoff timestamp is skipped and the prior round's price is used. If Midas publishes NAV at or near the cutoff hour, the cohort locks an older rate instead of that cutoff print. The locked price therefore depends on second-level timing between oracle publication and bucket boundaries.
Recommendations	If exact-cutoff prints should count, adjust the boundary condition so rounds at the cutoff timestamp are included. Otherwise, document that cohort pricing uses the last Chainlink round strictly before the bucket cutoff, not the first at or after it, and confirm this matches Midas's intended redemption accounting.
Comments / Resolution	Acknowledged.

Issue_08	<code>totalAssets</code> can revert even when there is no <code>mGLOBAL</code> exposure
Severity	Informational
Description	<code>CutoffMidasAccount.totalAssets()</code> always reaches <code>_pendingAssets()</code>. Inside <code>_pendingAssets()</code>, the contract validates the current Midas oracle price before checking whether there are any pending requests. This means <code>totalAssets()</code> can revert if the current <code>mGLOBAL</code> oracle is stale, unavailable, or outside bounds, even when the account has no <code>mGLOBAL</code> balance and no pending redemption requests. This can unnecessarily break accounting reads for an account that has no exposure requiring an oracle price.
Recommendations	Only validate the oracle when there is a token balance or pending request that actually needs pricing.
Comments / Resolution	Fixed.

Issue_09	A single invalid datapoint can brick the entire contract
Severity	Informational
Description	In <code>_pendingAssets</code>, the loop iterates over multiple rounds and checks if it has a valid price. <pre> while (timestamp >= nextBucketTimestamp) { [, answer,, timestamp,] = AggregatorV3Interface[aggregator].getRoundData[--roundId]; } if (answer <= 0) { revert InvalidCutoffPrice(); } </pre> The issue is that if a single round has price = 0, it can immediately break the contract, since the execution will revert with <code>InvalidCutoffPrice</code>. This can happen if the walk eventually reads invalid or empty rounds.
Recommendations	Consider ignoring an invalid price and iterating more until a valid price is found, within a bound.
Comments / Resolution	Acknowledged.

Issue_10	Direct token transfers affect accounting
Severity	Informational
Description	<code>CutoffMidasAccount</code> inherits <code>Account.totalAssets()</code>, which counts the account's raw token balances directly. Because of this, if <code>USDC</code> or <code>mGLOBAL</code> is sent directly to the account, those tokens are immediately included in <code>totalAssets()</code>, even though they did not enter through the intended flow. This can affect the reported account value and may make accounting harder to reason about.
Recommendations	Document that direct token transfers to the account affect <code>totalAssets()</code> .
Comments / Resolution	Acknowledged.

CutoffAccount

The `CutoffAccount` contract is a simple contract implementing the functions `currentBucket` and `nextCutoff`. `currentBucket` returns the bucket number corresponding to a timestamp, and `nextCutoff` returns the starting timestamp of the bucket that will start after the current active one. This contract is inherited by the `CutoffMidasAccount` contract.

No issues were identified in this contract.

3.2 Scope 2

Project	Symbiotic – Async/Account – Audit Report
Github repository	https://github.com/symbioticfi/core-mirror/blob/5752d056ba8de336d6f0b9716aef4a7b78b8a73b/src/contracts/adapters/ThreeFAdapter.sol
Resolution 1	https://github.com/symbioticfi/core-mirror/blob/d18d4699c6a48f1efad8d7904a6e3427cd0dfd5a/src/contracts/adapters/ThreeFAdapter.sol

ThreeFAdapter

The `ThreeFAdapter` contract is a `VaultV2` adapter that deploys vault assets into whitelisted 3F bridge facilitator requests. It signs offers as an ERC-1271 signer through `isValidSignature` using the configurable `offerSigner`. `onRequestConsumed` is the whitelisted request callback that enforces per-request bounds and minimum yield, then pulls principal from the vault via `allocateExact` inside a transient window and approves the request. `finalizeRequest` permissionlessly burns the adapter's PT and YT shares in a matured request back into assets and drops the request from tracking. `totalAssets` values live requests, `getMaxAssets` reports remaining deployable capacity, and the owner sets the signer and limits. The live principal is illiquid until finalization, so `_deallocate` returns zero.

Appendix: Just-in-time allocation

Allocation is driven by the request rather than by the delegator. During `onRequestConsumed` the adapter sets the transient flag `_inConsume`, which switches allocatable from zero to the base implementation, calls `allocateExact` on the delegator to pull exactly the principal from the vault, then clears the flag and approves the calling request for the principal. Outside this window, the adapter cannot receive vault allocations at all.

Privileged Functions

- `setOfferSigner`
- `setLimitsPerRequest`

Invariants

- `allocatable` is zero except inside the `_inConsume` window opened during `onRequestConsumed`.
- Each live request appears exactly once in the requests array, and `requestIndex` maps it to its array position plus one; a zero index means the address is not a tracked request.
- `requests.length` never exceeds `MAX_REQUESTS`.
- Every accepted request satisfies `minAssetsPerRequest <= principal <= maxAssetsPerRequest`, and its yield scaled by `YIELD_PRECISION` over principal is at least `minYieldPerRequest`.
- `pendingAssets` for a request is set when the request is consumed and cleared to zero when it is finalized or when the request address is not tracked.
- Principal deployed into a live request cannot be deallocated back to the vault; `deallocate` only ever returns assets that are free in the adapter.

Issue_11	<code>totalAssets</code> jumps when <code>canWithdraw</code> flips
Severity	Medium
Description	When a request is funded, the contract stores its value in <code>pendingAssets</code>, equal to the amount funded. It values the YT tokens at 0. The same is done in the request's <code>convertToAssets</code> function. When the Request's loan is repaid, the <code>convertToAssets</code> function now evaluates the PT and YT tokens at the actual repayment rate. This causes a sudden change in the <code>totalAssets</code> value the moment the <code>canWithdraw</code> flag in the request contract is flipped. Thus, all the yield in the Adapter is accrued in a single transaction. This makes the vault susceptible to sandwich attacks. Users can frontrun the <code>canWithdraw</code> flip to deposit funds into the vault at a lower asset-per-share and then withdraw immediately after at a higher asset-per-share, and capture most of the yield generated.
Recommendations	Consider implementing a way to drip the yield into the contract gradually after finalization, instead of realizing the profit in a single transaction.
Comments / Resolution	Acknowledged.

Issue_12	Matured 3F request value is counted in NAV, but is unavailable for withdrawals until finalization
Severity	Low
Description	Once a 3F request becomes withdrawable, <code>ThreeFAdapter.totalAssets</code> values the request through the adapter's PT/YT balances and the request's <code>convertToAssets</code> result. At that point, the matured request value is included in vault NAV. However, the value is still not available as vault liquidity. <code>ThreeFAdapter._deallocate</code> always returns 0, so the delegator cannot pull matured request value back to the vault through normal deallocation. The assets only become adapter-held free assets after someone calls <code>finalizeRequest</code>, which burns the adapter's PT/YT position in the request. This creates a mismatch between accounting and liquidity. The vault can include matured 3F value in <code>totalAssets</code>, while <code>withdrawable</code> cannot use that value to satisfy withdrawals.
Recommendations	Consider acknowledging and documenting this behaviour, or consider adding the <code>finalizeRequest</code> to the deallocation flow.
Comments / Resolution	Acknowledged.

Issue_13	Loan defaults can be bypassed by frontrunning finalizeWithdraw calls
Severity	Low
Description	When <code>finalizeRequest</code> is called, the contract redeems the PT and YT tokens into the base asset. Normally, this is profitable, but in case of defaults, this can evaluate to less than <code>pendingAssets</code> amount of assets. Since this loss is realized in a single transaction, users can bypass this loss by frontrunning this call and withdrawing assets before this call, forcing the other holders to bear this loss instead.
Recommendations	Consider adding a delay in vault withdrawals or acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_14	<code>getMaxAssets</code> ignores per-request principal limits, and request length limit
Severity	Low
Description	<code>getMaxAssets</code> is meant to tell 3F requestors the largest principal amount that can be funded into a new request right now. It only considers delegator headroom and vault withdrawable liquidity. It does not apply <code>maxAssetsPerRequest</code> or <code>minAssetsPerRequest</code>, even though <code>onRequestConsumed</code> enforces both on every consume. That mismatch breaks the function's stated contract. A requestor that sizes an offer from <code>getMaxAssets</code> can still revert on consume for reasons the view never signals. If capacity is 50 but <code>minAssetsPerRequest</code> is 100, <code>getMaxAssets</code> returns 50, even though a request of that size is not possible. Furthermore, if the <code>requests.length</code> matches <code>MAX_REQUESTS</code> already; no further requests can be consumed, irrespective of what <code>getMaxAssets</code> returns.
Recommendations	Update <code>getMaxAssets</code> to apply the same per-request bounds used in <code>onRequestConsumed</code> . Cap the result at <code>maxAssetsPerRequest</code> . If the capped value is below <code>minAssetsPerRequest</code> , or if <code>requests</code> is already at the max limit, return 0 so callers know no valid request size is available.
Comments / Resolution	Acknowledged, documented.

Issue_15	Request finalization accepts zero or partial recovery without checks or loss reporting
Severity	Informational
Description	<code>finalizeRequest</code> calls <code>burnAll</code> on the 3F request but ignores the returned principal and yield amounts. It always clears <code>pendingAssets</code>, removes the request from tracking, and emits <code>FinalizeRequest</code> with no recovered amount. The adapter never checks whether recovery is below the original principal, whether nothing was returned at all, or whether finalization left the adapter with the expected assets. If a 3F request defaults or repays less than expected, finalization still succeeds. The vault keeps no on-chain record of how much was lost.
Recommendations	Consider documenting and acknowledging this issue, or reverting and requiring explicit handling in case of a shortfall during finalization.
Comments / Resolution	Acknowledged.

Issue_16	Zero-principal requests can consume adapter request slots
Severity	Informational
Description	<code>onRequestConsumed</code> only rejected requests where <code>principalAssets</code> is below <code>minAssetsPerRequest</code> . If <code>minAssetsPerRequest</code> is set to zero, a whitelisted request can consume with <code>principalAssets</code> = 0. The yield check is skipped when the <code>principalAssets</code> is zero, but the request is still added to requests and consumes one of the <code>MAX_REQUESTS</code> slots. This can let zero-principal requests fill adapter capacity until they are finalized.
Recommendations	Rejects zero-principal requests explicitly, or require <code>minAssetsPerRequest</code> to be greater than zero.
Comments / Resolution	Acknowledged.

Issue_17	<code>getMaxAssets</code> triggers permissionless <code>sweepPending</code> side effects
Severity	Informational
Description	<code>ThreeFAdapter.getMaxAssets</code> looks like a capacity query but is not read-only. Every call runs <code>UniversalDelegator.sweepPending</code>, which processes the vault's pending withdrawal queue: sweeping free assets from adapters, filling the queue, and requesting deallocation from others as needed. Because <code>sweepPending</code> is permissionless, any account can trigger this sweep by calling <code>getMaxAssets</code>. Integrators using <code>staticcall</code> will revert on a state-changing call. Those using a normal call mutate vault state while only trying to read capacity.
Recommendations	Document that <code>getMaxAssets</code> is state-mutating and must not be called via <code>staticcall</code> .
Comments / Resolution	Acknowledged, documented.

Issue_18	Offer signer rotation invalidates pending offers with no grace period
Severity	Informational
Description	The owner can change <code>offerSigner</code> at any time via <code>setOfferSigner</code>, with no delay or overlap window. Offers must be signed by the current signer, and the 3F Request contract checks that inside consume before calling <code>onRequestConsumed</code>. That check and callback run in the same transaction, so <code>offerSigner</code> cannot change between them on-chain. The issue is rotation timing: offers signed under the old key but not yet consumed will fail once the signer is updated. When consume runs, validation uses the new signer. Old signatures revert with an invalid signature error before <code>onRequestConsumed</code> runs. No funds move, but the offer cannot be completed.
Recommendations	Consider timelocking <code>setOfferSigner</code> and clearing pending offers before rotating. Document that rotation immediately invalidates unconsumed offers signed under the previous key.
Comments / Resolution	Acknowledged, documented.

Issue_19	Stale offers fail when limits change without a transition period
Severity	Informational
Description	Off-chain 3F tooling sizes signed offers using <code>getMaxAssets</code> and the adapter's per-request limits. Both read live state: delegator headroom, vault withdrawable balance, sweep status, and current min/max/yield settings. All of these can change instantly, invalidating any in-flight signed offers instantly. The owner can call <code>setLimitsPerRequest</code>, the delegator admin can call <code>setLimits</code>, and capacity can shrink when other requests fill, liquidity drops, or a sweep is pending. There is no timelock or pending-limit state. If the pre-signed offers are used, it will lead to a revert.
Recommendations	Consider adding a timelock for changing the parameters, or managing the coordination of this off-chain.
Comments / Resolution	Acknowledged.

Issue_20	Adapter rejects compact ERC-2098 EOA signatures accepted by 3F request contracts
Severity	Informational
Description	ThreeFAdapter.isValidSignature validates offers by delegating to OpenZeppelin SignatureChecker.isValidSignatureNow(offerSigner, hash, signature). In this checkout, the OpenZeppelin EOA path calls ECDSA.tryRecover(hash, bytes signature), which only accepts 65-byte ECDSA signatures. The 3F request contracts use Solady's SignatureCheckerLib, whose EOA path accepts both standard 65-byte signatures and 64-byte ERC-2098 compact signatures. When the adapter is the offer.maker, the request validates the offer through the adapter's ERC-1271 isValidSignature function. As a result, a compact 64-byte signature can be rejected by the adapter.
Recommendations	Consider supporting ERC-2098 compact signatures in ThreeFAdapter.isValidSignature , by using the same checker as that of 3F. Alternatively, document that the 3F adapter only supports 65-byte EOA signatures.
Comments / Resolution	Acknowledged, documented.

Issue_21	Adapter cannot selectively cancel its own 3F offer nonces
Severity	Informational
Description	3F request contracts support offer cancellation through <code>setNonce(newNonce)</code>. The nonce is keyed by <code>msg.sender</code>, meaning only the maker address can advance its own nonce. For offers where <code>ThreeFAdapter</code> is the maker, the nonce owner is the adapter contract itself. The adapter exposes <code>setOfferSigner</code>, but it does not expose any function that forwards a nonce update to a 3F request contract. This prevents selective cancellation of a specific pending offer nonce for a specific request through the adapter. Operators must rely on broader controls such as rotating <code>offerSigner</code>, setting the signer to zero, changing request limits, or waiting for offer expiration.
Recommendations	Add an owner-only adapter function that calls <code>setNonce(newNonce)</code> on a whitelisted 3F request. Validate that the target request is whitelisted and uses the vault asset. Document signer rotation as a coarse emergency cancellation path, not a replacement for request-specific nonce management.
Comments / Resolution	Fixed following recommendations.

Issue_22	setRequestNonce cannot cancel offers while request is paused or not whitelisted
Severity	Low
Description	setRequestNonce reuses _validateRequest which only accepts requests with active whitelisted status and matching asset. If a request is paused or removed from the whitelist, the adapter owner cannot advance the nonce for that request until it is enabled/whitelisted.
Recommendations	Consider allowing nonce cancellation for requests that are not enabled or whitelisted.
Comments / Resolution	

3.3 Scope 3

Project	Symbiotic – Async/Account – Audit Report
Github repository	https://github.com/symbioticfi/core-mirror/blob/42b4c2e5dc36ded4b0390534926a87e48ce40bfe/src/contracts/adapters/II-adapter/common/AsyncRedeemAccount.sol
Resolution 1	

AsyncRedeemAccount

The `AsyncRedeemAccount` contract is an abstract base account for integrating with ERC-7540 async redeem vaults, extending `CoolDownAccount` with request-based redemption handling. It tracks the identifiers of its open redemption requests in the `requestIds` array and resolves the target async redeem vault dynamically by querying the ERC-7575 share token for the vault associated with the account asset. `_totalAssets` values the account as the sum of all pending redeem requests converted to assets at current prices plus the claimable amount reported by `maxWithdraw` at fulfillment prices.

Invariants

- Every id in `requestIds` has a nonzero `pendingRedeemRequest` in the vault after `_finalizeRequests` runs
- `requestIds` contains no duplicate request ids
- After `_requestRedeem`, the account's entire token-to-redeem balance has been submitted to the vault for redemption
- `_totalAssets` equals the asset value of all pending requests plus the currently claimable withdraw amount

Issue_23	<code>_finalizeRequest</code> can pop a fulfilled redemption without claiming it, stranding assets from the automated flow
Severity	Medium
Description	In <code>_finalizeRequests</code>, the <code>maxRedeem</code> function is called, and if <code>>0</code>, <code>redeem</code> is called to claim the redeemed assets. <pre>uint256 maxRedeem = IAsyncRedeemVault(asyncRedeemVault).maxRedeem(address(this)); if (maxRedeem > 0) { IAsyncRedeemVault(asyncRedeemVault).redeem(maxRedeem, address(this), address(this)); }</pre> After this, the redemption request is popped if <code>pending</code> is 0. The issue is that under certain conditions, <code>maxRedeem</code> can return 0. This happens when the admin of Centrifuge pauses token transfers via the hooks. <code>maxRedeem</code> calls <code>_canTransfer</code>, which can be used to disable this function. <pre>if (!_canTransfer(vault_, user, address(0), shares)) return 0;</pre> In that case, the request will be popped without redeeming the funds, since <code>maxRedeem = 0</code>. Future <code>_finalizeRequests</code> calls will also fail to redeem funds due to the early exit in case of an empty requests array. <pre>uint256 length = requestIds.length; if (length == 0) { return; }</pre> So if the Centrifuge vault is ever paused and then <code>sync</code> is called in that state, requests that have been redeemed but not claimed can get popped and then be unable to be claimed. This will persist until there are fresh funds that can be redeemed in the account, and the request ID gets added again in <code>_requestRedeem</code>, resulting in an availability DOS.

Recommendations	Consider allowing redemptions to process even when requestIds are empty, since in Centrifuge, all requests are fungible. Otherwise, add an admin-gated redeem function to redeem positions in case requestId is empty but positions exist.
Comments / Resolution	

Issue_24	Pricing inconsistency in <code>_totalAssets</code>
Severity	Low
Description	In <code>totalAssets</code>, the parent Account contract values the vault tokens using <code>IOracle[ORACLE].getPrice()</code>. <pre>function _tokenToRedeemToAssets(uint256 amount, uint256 rate) internal view returns (uint256) { return amount > 0 ? amount.mulDiv(rate * _unit, TO_ASSETS_DIVISOR) : 0; }</pre> The pending requests are, however, valued using <code>convertToAssets</code> directly. <pre>assets += IAsyncRedeemVault(asyncRedeemVault) .convertToAssets(IAsyncRedeemVault(asyncRedeemVault).pendingRed eemRequest(requestIds[i], address(this)));</pre> This creates an inconsistency, since the oracle price implements a min-max price gate, which is skipped when using raw <code>convertToAssets</code>.
Recommendations	Consider using the oracle price for both components.
Comments / Resolution	

Issue_25	The account cannot deal with cancelled redemptions
Severity	Low
Description	Centrifuge vault expects users to call <code>claimCancelRedeemRequest</code> if part of their redemption is cancelled. The account, however, does not implement any way to do so.
Recommendations	Consider adding an admin-gated function to call <code>claimCancelRedeemRequest</code> .
Comments / Resolution	

Issue_26	Centrifuge redeem can leave dust amounts of funds unprocessed
Severity	Low
Description	The Centrifuge contract natspec here warns that when using <code>redeem</code>, <i>When claiming redemption requests using <code>redeem()</code>, there can be some precision loss leading to dust. It is recommended to use <code>withdraw()</code> to claim redemption requests instead.</i> Thus, using <code>redeem</code> can lead to dust funds being left in the vault.
Recommendations	Consider following the recommendation and using <code>withdraw</code> instead.
Comments / Resolution	

Issue_27	Unsupported ERC7540 operations
Severity	Informational
Description	The contract states that it is built to support ERC7540 vaults. However, certain operations in the contract are only applicable to Centrifuge vaults, and don't actually follow the ERC 7540 vault standard. 1. <code>maxWithdraw</code> in ERC7540 inherits the definition from ERC4626, representing the maximum amount that can be redeemed by the user. However, in Centrifuge and the account contract here, <code>maxWithdraw</code> is expected to represent the redeemed but not yet claimed amount. 2. <code>_asyncRedeemVault</code> defined in the contract is only applicable for ERC7575 multi-currency vaults. It is not applicable for base ERC7540 vaults.
Recommendations	Consider either documenting that this contract is for Centrifuge only, or implementing ERC7540 spec functions here and then overriding them in <code>CentrifugeAccount</code> .
Comments / Resolution	

AsyncRedeemOracle

The `AsyncRedeemOracle` contract prices the shares of an ERC-7540 async redeem vault in 1e18 precision, extending the bounded Oracle base, which rejects any price outside its immutable `MIN_PRICE` and `MAX_PRICE` range. It serves as the price source for accounts in the II-adapter system that hold the vault's share token as their token to redeem.

Invariants

- `getPrice` always returns a value within `[MIN_PRICE, MAX_PRICE]` or reverts
- The returned price is expressed in 1e18 precision regardless of the share and asset token decimals
- `ASYNC_REDEEM_VAULT`, `SHARE_UNIT`, and `ASSET_UNIT` are fixed at deployment and never change